TERM-II PROJECT REPORT

1.Title Page

Course Name: PCCCS495 – Object Oriented Programming

Project Title: Multithreaded Chat Application using Observer Pattern

Student Name: **Jayashri Rout**

Roll Number: 09

Instructor Name: Soumadip Biswas, Swagatam Basu, Susovan

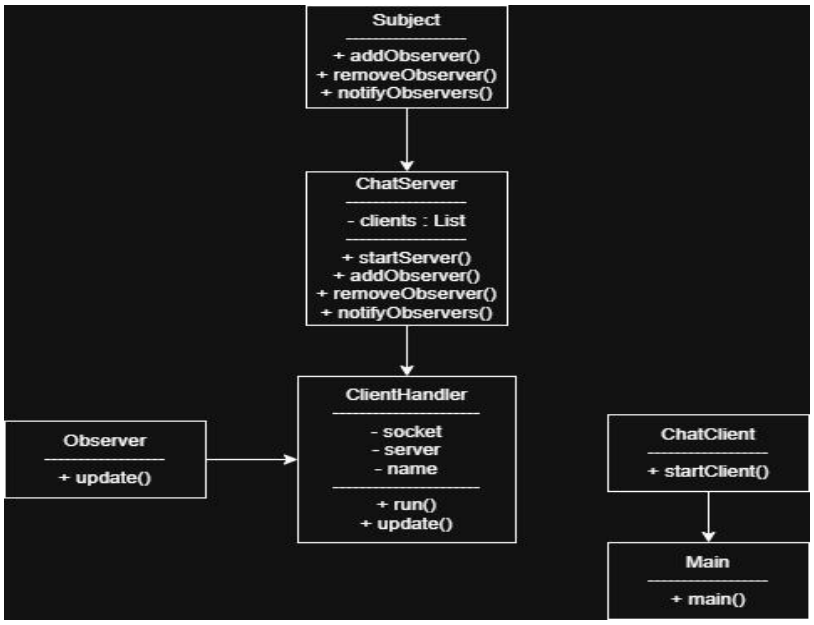
Semester: Spring 2026

2.Problem Statement

In modern communication systems, real-time interaction between multiple users is essential. Traditional single-threaded applications are not capable of efficiently handling multiple client connections simultaneously. This leads to delays, blocking issues, and poor user experience.

The objective of this project is to develop a chat application that allows multiple users to communicate simultaneously over a network. The system uses multithreading to handle multiple clients concurrently and the Observer Design Pattern to efficiently broadcast messages to all connected users. This ensures smooth communication and scalability of the system.

3.System Design



UML Class Diagram

Description of Major Classes

• ChatServer:

Handles incoming client connections and manages communication between clients.

• ClientHandler:

Handles individual client communication using separate threads.

• Observer Interface:

Defines the update() method for receiving messages.

• Subject Interface:

Defines methods to manage observers (clients).

• ChatClient:

Client-side program for sending and receiving messages.

• Main:

Entry point to run server or client.

Package structure

```
Src/
 ├── ChatServer.java
 ├── ClientHandler.java
 ├── ChatClient.java
 ├── Observer.java
 ├── Subject.java
 ├── Main.java
```

Interaction Flow

- Client connects to server
- Server creates a new thread for the client
- Client sends message
- Server receives message

- Server broadcasts message to all clients

4.OOP Concepts Demonstrated

Abstraction

Where used: Observer & Subject interfaces

Why used: To define common behavior without implementation

Code snippet :

```
Public interface Observer {
  Void update(String message);
```

Explanation:

This interface defines a contract for all observers (clients) to receive updates.

Inheritance

Where used: ClientHandler implements Observer

Code snippet:

```
Public class ClientHandler implements Runnable, Observer {
```

Explanation:

ClientHandler inherits behavior from Observer interface.

Polymorphism

Where used: update() method

Code snippet:

```
@Override
Public void update(String message) {
  Out.println(message);
}
```

Explanation:

Different clients receive messages using the same method.

Exception Handling

Where used: Socket operations

Code snippet:

```
Try {
  Socket socket = new Socket("localhost", 5000);
} catch (Exception e) {
  e.printStackTrace();
}
```

Explanation:

Ensures program does not crash due to runtime errors.

Collections / Threads

Where used: ArrayList + TThread

Code snippet:

```
List<Observer> clients = new ArrayList<>();
Thread t = new Thread(handler);
t.start();
```

Explanation:

List stores clients and threads allow concurrent execution.

5.Implementation Highlights

Snippet 1: Server Socket

```
ServerSocket server = new ServerSocket(5000);
Socket socket = server.accept();
```

Justification:

Allows server to listen and accept client connections.

Snippet 2: Multithreading

```
Thread t = new Thread(handler);
t.start();
```

Justification:

Each client runs independently.

Snippet 3: Message Broadcasting

Justification:

```
for (Observer o : clients) {
    o.update(message);
}
```

Implements Observer pattern for communication.

6. Testing & Error Handling

Test Cases

- Single client connection ✅
- Multiple client connections ✅
- Message broadcasting ✅
- Client disconnection ✅

Edge Cases

- Client disconnects unexpectedly
- Empty message input
- Multiple clients sending simultaneously

Failure Scenarios

- Server not running → client fails to connect
- Invalid input → handled via exception

7. Git Workflow

Explanation

```
PS C:\Users\Jayashri Rout\OneDrive\Desktop\term-ii-project-submission-Jayashri-rout> git log --oneline
cbce6d6 (HEAD -> main, origin/main, origin/HEAD) Fixed all the Bugs
9acebea Fixed null pointer exception by properly initializing server in ClientHandler constructor
550ace3 added main entry point for running server and client
0b53e0b implemented chat client for sendind and receiving messages
007acc2 added user join and leave notifications
3b6c144 enabled message broadcasting from clients to all users
b69add0 implemented observer pattern for message broadcasting
43e298b implemented observer interface in client handler
57a5923 Integrated multithreading in chat server to handle multiple clients
33d9870 created client handler thread skeleton
1ed6ab6 Implemented basic chat server socket setup
bea422c added subject interface for observer pattern
0a654d0 created project structure and added observer interface
1333163 add deadline
59939a0 Initial commit
```

1. Initial setup commits
2. Step-by-step feature addition
3. Multithreading implementation
4. Observer pattern integration
5. Bug fixing (NullPointerException)
6. Final documentation

This shows incremental development.

8. Conclusion & Future Scope

Conclusion

This project successfully demonstrates a multithreaded chat application using the Observer pattern. It efficiently handles multiple clients and enables real-time communication.

Future Scope

- GUI-based chat interface
- Private messaging
- Chat history storage
- File sharing
- Encryption for security